

LP V Assignment No 1

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

```
In [1]: import pandas as pd
import numpy as np

data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)

data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

columns = [
    "CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM", "AGE",
    "DIS", "RAD", "TAX", "PTRATIO", "B", "LSTAT"
]

boston = pd.DataFrame(data, columns=columns)
boston["PRICE"] = target

# Save for reuse
boston.to_csv("boston_house_prices.csv", index=False)
```

```
In [2]: pip install numpy pandas scikit-learn tensorflow matplotlib
```

```
Requirement already satisfied: numpy in /Users/sangambhure/jupyter-l
angchain-311/lib/python3.11/site-packages (1.26.4)
Requirement already satisfied: pandas in /Users/sangambhure/jupyter-l
angchain-311/lib/python3.11/site-packages (2.3.3)
Requirement already satisfied: scikit-learn in /Users/sangambhure/ju
pyter-langchain-311/lib/python3.11/site-packages (1.8.0)
Requirement already satisfied: tensorflow in /Users/sangambhure/jupy
ter-langchain-311/lib/python3.11/site-packages (2.21.0)
Requirement already satisfied: matplotlib in /Users/sangambhure/jupy
ter-langchain-311/lib/python3.11/site-packages (3.10.8)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/sang
ambhure/jupyter-langchain-311/lib/python3.11/site-packages (from pan
das) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/sangambhure/ju
pyter-langchain-311/lib/python3.11/site-packages (from pandas) (202
5.2)
Requirement already satisfied: tzdata>=2022.7 in /Users/sangambhure/
jupyter-langchain-311/lib/python3.11/site-packages (from pandas) (20
25.3)
Requirement already satisfied: scipy>=1.10.0 in /Users/sangambhure/j
upyter-langchain-311/lib/python3.11/site-packages (from scikit-lear
n) (1.17.0)
Requirement already satisfied: joblib>=1.3.0 in /Users/sangambhure/j
upyter-langchain-311/lib/python3.11/site-packages (from scikit-lear
n) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Users/sangam
```

```

In [3]: # -----
# Import Libraries
# -----
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# -----
# Load Dataset
# -----
boston = pd.read_csv("boston_house_prices.csv")

# Features and Target
X = boston[['LSTAT', 'RM', 'PTRATIO']]
y = boston['PRICE']

# -----
# Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=4
)

# -----
# Feature Scaling
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Linear Regression Model
# -----
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

y_pred_lr = lr_model.predict(X_test_scaled)

print("◆ Linear Regression Model Evaluation:")
print("MSE:", mean_squared_error(y_test, y_pred_lr))
print("MAE:", mean_absolute_error(y_test, y_pred_lr))
print("R2 Score:", r2_score(y_test, y_pred_lr))

# -----

```

```

# Neural Network Model
# -----
model = Sequential([
    Input(shape=(3,)),          # FIXED (No warning now)
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

# -----
# Train Neural Network
# -----
history = model.fit(
    X_train_scaled,
    y_train,
    epochs=50,
    validation_split=0.05,
    verbose=1
)

# -----
# Evaluate Neural Network
# -----
y_pred_nn = model.predict(X_test_scaled)
mse_nn, mae_nn = model.evaluate(X_test_scaled, y_test, verbose=0)

print("\n◆ Neural Network Model Evaluation:")
print("MSE:", mse_nn)
print("MAE:", mae_nn)

# -----
# Prediction (FIXED)
# -----
new_data = pd.DataFrame(
    [[0.1, 10.0, 5.0]],
    columns=['LSTAT', 'RM', 'PTRATIO']  # FIXED (no warning)
)

new_data_scaled = scaler.transform(new_data)

prediction = model.predict(new_data_scaled)

print("\n🏠 Predicted House Price:", prediction[0][0])

```

◆ Linear Regression Model Evaluation:

MSE: 30.340105190234606

MAE: 3.5844321029226935

R2 Score: 0.6733732528519258

Epoch 1/50

```

- val_loss: 7.0283 - val_mae: 2.3538
Epoch 39/50
12/12  0s 2ms/step - loss: 17.1547 - mae: 2.7523
- val_loss: 9.3732 - val_mae: 2.6403
Epoch 40/50
12/12  0s 2ms/step - loss: 16.2928 - mae: 2.7263
- val_loss: 7.6513 - val_mae: 2.4726
Epoch 41/50
12/12  0s 2ms/step - loss: 16.2670 - mae: 2.7355
- val_loss: 7.6302 - val_mae: 2.4696
Epoch 42/50
12/12  0s 2ms/step - loss: 15.9728 - mae: 2.6787
- val_loss: 7.9476 - val_mae: 2.4932
Epoch 43/50
12/12  0s 2ms/step - loss: 15.8358 - mae: 2.6736
- val_loss: 7.4379 - val_mae: 2.4478
Epoch 44/50
12/12  0s 2ms/step - loss: 15.6467 - mae: 2.6483
- val_loss: 7.7210 - val_mae: 2.4734
Epoch 45/50
12/12  0s 2ms/step - loss: 15.6570 - mae: 2.6631
- val_loss: 7.8939 - val_mae: 2.4873
Epoch 46/50
12/12  0s 2ms/step - loss: 15.5639 - mae: 2.6342
- val_loss: 7.2356 - val_mae: 2.4186
Epoch 47/50
12/12  0s 2ms/step - loss: 15.5912 - mae: 2.6601
- val_loss: 8.7670 - val_mae: 2.5856
Epoch 48/50
12/12  0s 2ms/step - loss: 15.6560 - mae: 2.6761
- val_loss: 7.5255 - val_mae: 2.4586
Epoch 49/50
12/12  0s 2ms/step - loss: 15.5813 - mae: 2.7117
- val_loss: 7.1780 - val_mae: 2.4190
Epoch 50/50
12/12  0s 3ms/step - loss: 15.4834 - mae: 2.6116
- val_loss: 8.3824 - val_mae: 2.5675
4/4  0s 8ms/step

```

◆ Neural Network Model Evaluation:

MSE: 20.953916549682617

MAE: 2.835590124130249

1/1  0s 13ms/step



Predicted House Price: 84.05499

In []: